

Tutorial de código Java

Índice

Fundamentos de código Java	2
Palabras reservadas.....	3
Package.....	4
Import.....	4
Tipos de datos.....	5
Tipos de datos “referencia” u “objeto”	8
Operadores.....	9
Estructuras de control	10
Control de flujo. Bucles	10
Control de flujo: condicionales.....	11

Fundamentos de código Java

Cada programa Java debe observar al menos las siguientes reglas:

- Importan las mayúsculas y minúsculas
- Los bloques de código se encierran entre { y }
- Un programa java, debe tener al menos un método principal, comúnmente llamado “main”

```
public class Prueba {  
    public static void main (String [] args) {  
        ...  
    }  
}
```

- Cada instrucción en Java termina con un “;”
- No se pueden utilizar espacios entre los denominadores, tampoco guiones medios (“-”), no se pueden utilizar acentos, eñe, ni otros caracteres especiales. Cabe aclarar que, si bien no se recomienda, sí se pueden usar guiones bajos (“_”)
- Los denominadores no pueden comenzar con un número
- Los comentarios pueden ser de una línea o multi-línea
 - Los de una línea se hacen con “//”
 - Los multi línea se encierran entre “/*” y “*/”

Cada programa Java, no necesariamente debe, pero es muy importante que respete las siguientes reglas:

- En código fuente de cada unidad de software (“clase”) en Java, va escrito en un archivo de texto plano de igual nombre, respetando mayúsculas y minúsculas y tendrá la extensión “.java”.
- Los nombres de las clases comienzan con mayúsculas, y cada palabra nueva, agrega una mayúscula. A esto se lo conoce como “camelCase” (o más

estrictamente como “PascalCase”, debido a que fue introducido originalmente en el lenguaje de programación Pascal). Por ejemplo:

```
public class AutoDeportivo {  
...  
}
```

- Los nombres de las variables comienzan con minúsculas, y cada palabra nueva comienza con una mayúscula (esto se conoce como “camelCase”).

```
int estaEsUnaVariable = 3;
```

- Los nombres de las operaciones, al igual que las variables, comienzan con minúsculas y cada palabra nueva lleva mayúsculas. Los argumentos se nombran de igual manera que las variables.

Palabras reservadas

La siguiente es una lista completa de las palabras reservadas en Java, la mayoría de las cuales estarán cubiertas en la materia. Corresponden a tipos de variables, instrucciones, etc.:

abstract	assert	boolean	break
byte	case	catch	char
class	const	continue	default
do	double	else	enum
extends	final	finally	float
for	goto	if	implements

<code>import</code>	<code>instanceof</code>	<code>int</code>	<code>interface</code>
<code>long</code>	<code>native</code>	<code>new</code>	<code>package</code>
<code>private</code>	<code>protected</code>	<code>public</code>	<code>return</code>
<code>short</code>	<code>static</code>	<code>strictfp</code>	<code>super</code>
<code>switch</code>	<code>synchronized</code>	<code>this</code>	<code>throw</code>
<code>throws</code>	<code>transient</code>	<code>try</code>	<code>void</code>
<code>volatile</code>	<code>while</code>		

Package

Las clases en java pueden organizarse en carpetas o directorios dentro del programa. Sabiendo que los archivos de código fuente son archivos de texto plano, los paquetes son solo directorios en el disco que contienen esos archivos.

Import

Cuando en la clase se usa un paquete y usamos clases que provienen de otros paquetes, debemos “importar” esas clases, mediante la sentencia “import” seguida del nombre de la clase, incluyendo el paquete al que pertenece. Los nombres de los paquetes utilizan camelCase. Por ejemplo:

```
import paquete1.otroPaquete.ClaseA;

public class Clase B {

    ClaseA.operacion1();

}
```

Tipos de datos

Java es un lenguaje fuertemente tipado, cada variable debe llevar obligatoriamente el tipo correspondiente. Sin embargo, puede haber tipos paramétricos, que ayudan a flexibilizar el contenido de las variables.

Los tipos de datos en java se pueden dividir en dos grandes grupos. Los llamados “primitivos” y los tipos de datos “Referencia”. El típico dato “primitivo” es el tipo de número entero: `int`.

Sin embargo, el tipo de dato entero “primitivo” también tiene un correlato en un tipo de dato “Referencia” (también llamado “Objeto”) denominado `Integer` (notar el uso de mayúsculas en este caso, dado que `Integer` es una clase). Esto ocurre con todos los tipos de datos primitivos:

Tipo Primitivo	Tipo Referencia
<code>int</code>	<code>Integer</code>
<code>long</code>	<code>Long</code>
<code>char</code>	<code>Character</code>
<code>byte</code>	<code>Byte</code>
<code>short</code>	<code>Short</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>
<code>boolean</code>	<code>Boolean</code>

Es por esto que preferiremos, para la materia el nombre “escalares” a los denominados “primitivos” para hacer una diferencia más marcada. Esto es porque para definir una variable de tipo `int`, alcanza con:

```
int x = 3;
```

Y para usar una variable de tipo Integer, necesito crear un objeto nuevo, y ya no es tan sencillo:

```
Integer x = new Integer(3);
```

El uso de uno o de otro dependerá del caso y su uso se deberá evaluar de acuerdo con el problema a resolver. Es importante notar que cuando los valores escalares tienen siempre un valor por default, los otros no, por ejemplo:

```
int x; // el valor de x es "0"  
Integer y; // y no tiene valor, es "nulo" (null)
```

Tipos de datos primitivos o escalares

A continuación, enumeraremos los tipos de valores “escalares”, también llamados a veces “literales” dado que necesitan de algún carácter extra para definir el tipo (char, long, float, double)

- byte:
 - Número entero, de 8 bits
 - Con signo
 - Valor por defecto: 0
- short:
 - Número entero de 16 bits
 - Con signo
 - Valor por defecto: 0
- int:
 - Número entero, de 32 bits
 - Con signo
 - Valor por defecto: 0
 - Cuando se define un valor entero de una variable, éste se asume como int
- long:
 - Número entero de 64 bits

- Con signo
- Valor por defecto: 0l
- Al definirlo, debe usarse la letra “l” para indicar al compilador que no se trata de un “int”
- Ejemplos:
 - `long f = 33344546456;` << no es válido, un valor entero se asume como “int”, pero este número es más grande que lo que puede almacenar un int
 - `long g = 33344546456l;` << al agregar la “l” indico al compilador que ese número debe ser interpretado como un long
- `float`:
 - Número de punto flotante, de 32 bits
 - Con signo
 - Valor por defecto: 0.0f
 - Al definirlo, debe usarse la letra “f” para indicar al compilador que el valor no debe ser interpretado como un `double`
 - Ocurre algo similar que con el `int` y el `long`; de acuerdo con valor debe agregarse el “f”.
- `double`
 - Número de punto flotante, de 64 bits
 - Con signo
 - Valor por defecto: 0.0d
 - Ocurre algo similar que con el `int` y el `long`; y de acuerdo con valor debe agregarse el “d”.
- `boolean`
 - Es un valor de un bit, verdadero o falso
 - Toma valores “true” o “false”
 - Valor por defecto: false

- `char`
 - Es un valor entero, de 16 bits
 - Sin signo
 - Va desde el 0 (carácter `\u0000`) al 65535 inclusivo (carácter `\uffff`)
 - Puede declararse directamente con comillas simples: `char x = 'p';`
 - Hay caracteres de control, para uso especial:

<code>\n</code>	Nueva línea (0x0a)
<code>\r</code>	Retorno de carro (0x0d)
<code>\f</code>	Alimentación (0x0c)
<code>\b</code>	Backspace (0x08)
<code>\s</code>	Espacio (0x20)
<code>\t</code>	tab
<code>\</code>	Escape
<code>\ddd</code>	Carácter Octal con valor (ddd)
<code>\uxxxx</code>	Carácter Hexadecimal con valor (xxxx)

Tipos de datos “referencia” u “objeto”

Son los obtenidos a partir de las clases. Cada variable de tipo “referencia” contendrá un objeto con el valor, obtenido mediante un constructor. Un arreglo (array) en Java es un tipo de dato en sí mismo y, dado que es un objeto, se lo puede considerar un tipo de dato “referencia”. El valor por defecto es siempre “null”, es decir, “nada”. Si bien hay un correlato para cada tipo primitivo, cualquier variable cuyo tipo sea un objeto es una variable de tipo “referencia”. Por ejemplo:

```
Integer i = new Integer(24);

Auto a = new Auto();
```


Tipo String

El tipo referencia “String” es como cualquier otro tipo de referencia (notar que su nombre comienza con mayúscula, porque String es una clase), pero es uno de los más utilizados para incluir cadenas de texto en el código. Los String se definen, a diferencia de los char, con comillas dobles. Por ejemplo:

```
String s = “esto es un texto”;
```

Si bien ‘s’ es una variable de tipo referencia String, y almacena un objeto de tipo String, no usamos un constructor (como hicimos más arriba con Integer, por ejemplo). Esto se debe a que el lenguaje incluye un “atajo” para crear nuevos objetos String de forma más sencilla. De otra manera, deberíamos crear cadenas de texto a partir de secuencias de caracteres individuales (de hecho, es posible crear Strings de esta manera), pero sería muy tedioso.

Operadores

El lenguaje Java provee varios operadores aritméticos, lógicos, de asignación, etc. En Java no es posible la sobrecarga de operadores.

Entre los aritméticos están los típicos de suma (+), resta (-), multiplicación (*), división (/), módulo (%), incremento (++) y decremento (--). Cabe aclarar que con el operador “+” también podemos concatenar (unir) una cadena de caracteres con otra.

A su vez, hay operadores relacionales o de comparación:

- `a == b`: verifica si a y b tienen igual valor.
- `a != b`: verifica si a y b tienen distinto valor.
- `a > b`, `a < b`: verifica si el valor de a es mayor/ menor que el de b.
- `a >= b`, `a <= b`: verifica si el valor de a es mayor o igual / menor o igual que el de b.

Los operadores lógicos comparan condiciones y resultan en valores booleanos:

- `&&`: es el comparador lógico AND.
- `||`: es el comparador lógico OR.
- `!`: sirve para negar una condición.

Los operadores de bit, como su nombre lo dice, hace operaciones a nivel de bits:

- `&`: AND bit a bit.
- `|`: OR bit a bit.
- `^`: XOR bit a bit.
- `~`: operador unario para invertir bits.
- `<<` y `>>`: shift de bits a izquierda y derecha respectivamente.
- `>>>`: shift a derecha, llenado de ceros .

El operador de asignación en Java es el `"="`. A su vez, hay otros combinan operaciones junto con la asignación:

- `+=` : suma y asigna, por ejemplo: `a += 3` es igual que hacer `a = a + 3`;
- `-=`: resta y asigna, de forma similar
- `*=`: multiplica y asigna
- `/=`: divide y asigna
- `%=`: calcula el módulo y asigna

Estructuras de control

Control de flujo. Bucles

Hay tres maneras de repetir una tarea y dos maneras adicionales de controlar esa repetición:

- `while`: repite las instrucciones mientras que se cumpla la condición. Aquí se evalúa la condición antes de ejecutar las operaciones.

```
while(condición) {  
    //instrucciones a repetir  
}
```

- for: repite instrucciones mientras que se cumpla la condición y administra variables para el control del bucle.

```
for(condInicial; condParaBucle; accionLuegoDelBucle) {
    //instrucciones a repetir
}
```

```
for(int x = 0; x<10; x++) {
    //instrucción que se repetirá 10 veces
}
```

- do...while: es similar al while, pero la evaluación de la condición se da luego de ejecutar las operaciones.

```
do {
    //instrucciones a repetir
} while (condición);
```

- continue: permite “saltar” la iteración y seguir con el siguiente paso del bucle.
- break: permite “romper” el bucle.

Control de flujo: condicionales

- if: similar al de otros lenguajes, el “if” toma una condición y ejecuta el código indicado si se cumple. Si se desea ejecutar otra porción de código si no se cumple la condición, se utiliza la sentencia “else”.

```
if(condición) {
    //código a ejecutar si condición es verdadera
} else {
    //código a ejecutar si condición es falsa
}
```

- switch: para evaluar una expresión, por múltiples y diferentes valores, en vez de usar una sucesión de if...else... podemos usar switch. La estructura es la siguiente:

```
switch(expresion) {  
    case expresión_vale_x:  
        //instrucciones a ejecutar si la expresión vale X  
    case expresión_vale_y:  
        //instrucciones a ejecutar si la expresión vale Y  
    ...  
    default:  
        // a ejecutar si la expr no vale ni X, ni Y, ni...  
}
```

- operador if ternario: sustituye una estructura if...else pero se puede escribir en una sola línea:

```
condición ? salidaPorVerdadero : salidaPorFalso  
  
x % 2 == 0 ? "numero par" : "numero impar";
```